

Offline & Online Anomaly Detection in Credit Card Data using Isolation Forest

Karina Osipova, Saidul Islam Nayan, and Matthew Balogh

Ophelia R&D, ELTE Data Science Club

1 Introduction

Fraudulent transactions related to credit cards either result in financial losses for companies or unjust charges to customers. Detecting these transactions is essential to minimize those outcomes. In this work, our goal is to detect fraudulent transactions in Credit Card data, in both offline and online setting, in which the data is given as a large batch (offline), or when it arrives incrementally (online). To do so, we aim to utilize the *Isolation Forest* algorithm and its stream mining counterpart *Half-Space Trees*. To support our choice, we formulate the following research questions. From the traditional machine learning point of view:

- What is an anomaly and anomaly detection?
- How does *Isolation Forest* detect anomalies?
- How does it differ from *Random Forest*?

and from the stream processing angle:

- Stream data characteristics, advantages and challenges
- How does *Half-Space Trees* relate to *Isolation Forest*?
- How does it differ from offline *Isolation Forest* version?

2 Background

2.1 Anomaly Detection

An *outlier* is an observation, a data point in a dataset that significantly differs from other observations which are considered as normal in a population of interest. The terms of outlier and anomaly are often used interchangeably both in literature and in practice. Besides, according to Aggarwal (2017), outliers are also referred to as *abnormalities*, *discordants*, *deviants*, or *anomalies*. As Grubbs (1969), Hawkins (1980), and Barnett, Lewis, et al. (1994) put it, such an observation (or subset of observations):

...appears to deviate markedly from other members of the sample in which it occurs (Grubbs 1969)

...deviates so much from other observations as to arouse suspicions that it was generated by a different mechanism (Hawkins 1980)

...appears to be inconsistent with the remainder of that set of data (Barnett, Lewis, et al. 1994)

2.2 What is Anomaly Detection

Anomaly detection is the process of identifying observations that deviate significantly from the expected or normal behavior within a dataset. Such observations, commonly referred to as anomalies, outliers, or deviations, often correspond to rare but critical events, such as fraudulent financial transactions, network intrusions, or system failures. Detecting these anomalous patterns is essential across domains including cybersecurity, finance, healthcare, and industrial monitoring. Traditional anomaly detection approaches include statistical methods such as Kernel Density Estimation, distance-based methods, and machine learning techniques such as one-class support vector machines (SVMs) and ensemble-based methods like Isolation Forest. These methods operate under the assumption that anomalies are rare and exhibit characteristics significantly different from normal observations. However, anomaly detection remains challenging due to several inherent factors. First, anomalies are typically rare, resulting in highly imbalanced datasets. Second, labeled anomaly data is often unavailable, limiting the applicability of supervised learning methods. Third, high-dimensional data increases complexity and reduces the effectiveness of distance-based approaches. Based on the availability of labeled data, anomaly detection methods can be categorized into three main types:

- **Supervised methods**, which require labeled examples of both normal and anomalous observations. These methods typically achieve high accuracy but are rarely applicable in real-world scenarios due to limited labeled anomaly data.
- **Semi-supervised methods**, which are trained primarily on normal data and detect deviations from learned normal patterns.
- **Unsupervised methods**, which require no labeled data and detect anomalies based on intrinsic properties of the dataset, such as density, distance, or isolation characteristics.

Isolation-based methods, such as Isolation Forest, belong to the unsupervised category and are particularly effective due to their scalability, efficiency, and ability to detect anomalies without explicit modeling of normal data distributions. These properties make them well suited for large-scale financial transaction data and streaming environments.

2.3 Random Forest

Random Forest is a widely used machine learning algorithm that belongs to the family of ensemble learning methods. Originally introduced and trademarked by Leo Breiman and Adele Cutler, the algorithm combines the predictions of multiple decision trees to produce a single, more accurate output. Due to its flexibility and robustness, Random Forest is commonly applied to both classification and regression problems across a wide range of domains.

At its core, the Random Forest algorithm is based on decision trees, which are supervised learning models that make predictions by recursively splitting data into subsets based on feature values. Each split corresponds to a decision node, while the final outcomes are represented as leaf nodes. The quality of each split is typically evaluated using metrics such as Gini impurity, information gain, or mean squared error. While decision trees are intuitive and easy to interpret, they are often prone to overfitting and high variance when trained on complex datasets.

Random Forest addresses these limitations by constructing an ensemble of decision trees and aggregating their predictions. This approach is based on the concept of bagging (bootstrap aggregation), where multiple training datasets are created by randomly sampling the original dataset with replacement. Each decision tree is trained independently on a different sample, and the final prediction is obtained by averaging the results (for regression) or by majority voting (for classification). This ensemble strategy reduces variance and improves the model's generalization performance.

In addition to bagging, Random Forest introduces feature randomness, also known as the random subspace method. Instead of evaluating all available features when splitting a node, each tree considers only a random subset of features. This process reduces correlation among the individual trees and further improves the stability and predictive performance of the model. By combining multiple weak learners that are relatively uncorrelated, Random Forest produces more reliable predictions while mitigating issues such as overfitting and bias that commonly affect single decision trees. Overall, Random Forest is valued for its strong predictive performance, ability to handle large and high-dimensional datasets, and resilience to noise in the data (Kavlakoglu 2026).

2.4 Isolation Forest

Isolation Forest is an unsupervised anomaly detection algorithm specifically designed to efficiently identify anomalies in large and high-dimensional datasets. Unlike traditional methods that attempt to model normal data points, Isolation Forest directly isolates anomalies through recursive partitioning of the data

space. This fundamental difference enables improved computational efficiency and scalability.

The key assumption behind Isolation Forest is that anomalous observations are both rare and distinct. As a result, they require fewer random partitions to be isolated compared to normal observations. The algorithm constructs an ensemble of binary trees, known as Isolation Trees, using randomly selected features and random split values. Each tree recursively partitions the dataset until individual data points are isolated. The number of splits required to isolate a data point corresponds to its *path length*. Since anomalies differ significantly from normal observations, they tend to be isolated closer to the root of the tree, resulting in shorter path lengths. In contrast, normal observations require more splits and therefore have longer path lengths. To improve robustness and stability, Isolation Forest aggregates results across multiple independently generated trees. The anomaly score of a data point is computed based on the average path length across all trees. Observations with shorter average path lengths are more likely to be anomalous, while observations with longer path lengths are considered normal.

Isolation Forest offers several important advantages:

- Computational efficiency with linear time complexity relative to dataset size
- No requirement for labeled training data
- Effectiveness in high-dimensional data
- Scalability to large datasets
- Robustness to noise and irrelevant features

These properties make Isolation Forest particularly suitable for financial fraud detection, where datasets are large, anomalies are rare, and real-time or near real-time detection may be required.

2.5 Anomaly Score Calculation

Isolation Forest determines the anomaly score of a data point based on its expected isolation path length. The path length, denoted as $h(x)$, represents the number of edges traversed from the root node to the leaf node that isolates observation x . Because path length depends on tree size, a normalization factor is required. This factor is based on the average path length of unsuccessful searches in Binary Search Trees, denoted as $c(n)$, where n represents the number of observations:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

where $H(n)$ is the harmonic number, approximated as:

$$H(n) \approx \ln(n) + \gamma$$

and γ is the Euler–Mascheroni constant. The anomaly score $s(x, n)$ is then computed as:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}}$$

where $E[h(x)]$ represents the average path length of observation x across all trees in the forest. The anomaly score ranges between 0 and 1 and can be interpreted as follows:

- If $s(x, n) \approx 1$, the observation is highly likely to be anomalous.
- If $s(x, n) \approx 0.5$, the observation is likely normal.
- If $s(x, n) \approx 0$, the observation is highly unlikely to be anomalous.

Thus, Isolation Forest identifies anomalies based on how quickly observations can be isolated relative to expected isolation depth.

2.6 Computational Complexity

Isolation Forest is computationally efficient compared to many traditional anomaly detection algorithms. Its training phase operates with a time complexity of $O(n \log n)$, where n represents the number of samples, while the prediction phase has a complexity of $O(n \log m)$, where m denotes the number of trees in the ensemble. This relatively low computational complexity enables the algorithm to scale effectively to large datasets. Additionally, the algorithm employs sub-sampling, which reduces memory usage and further improves computational efficiency. The independent construction of isolation trees also allows for parallel processing, making the algorithm well-suited for distributed and multi-core computing environments (DeepFA 2025).

2.7 Limitations of Isolation Forest

Despite its efficiency and widespread use, Isolation Forest presents several limitations. First, its performance is highly sensitive to hyperparameter selection, particularly the number of trees (*n_estimators*), subsample size (*max_samples*), and the contamination parameter that estimates the proportion of anomalies. Improper parameter tuning can significantly reduce detection accuracy.

Second, although the method is designed for imbalanced datasets, extreme imbalance scenarios (e.g., when anomalies constitute less than 0.1% of the data) may negatively affect performance and require careful adjustment of parameters. Another limitation is limited interpretability. While the algorithm provides an anomaly score indicating the likelihood of abnormality, it does not offer clear explanations for why a specific observation is classified as anomalous.

Furthermore, Isolation Forest is primarily effective at detecting global anomalies and may perform poorly when dealing with local anomalies, where observations appear normal within a local region but are anomalous in a broader context. Finally, the algorithm is sensitive to irrelevant or noisy features. Datasets containing many non-informative variables can degrade detection performance, making proper feature selection and preprocessing important steps before applying the algorithm (DeepFA 2025).

2.8 Random Forest vs. Isolation Forest

Although Random Forest and Isolation Forest share a similar conceptual foundation based on ensembles of decision trees, they are designed for different machine learning tasks and operate using fundamentally different principles. Random Forest is a supervised learning algorithm primarily used for classification and regression problems. It relies on labeled data to train multiple decision trees and aggregates their predictions through majority voting or averaging to improve accuracy and reduce variance.

In contrast, Isolation Forest is an unsupervised anomaly detection algorithm specifically designed to identify outliers within a dataset. Rather than predicting class labels or numerical outputs, it focuses on isolating anomalous observations through recursive random partitioning of the data. While Random Forest constructs trees to learn predictive relationships between features and target variables, Isolation Forest builds trees that isolate data points and measures how quickly a point can be separated from the rest of the dataset. Because anomalies are rare and structurally different from normal observations, they tend to have shorter isolation paths within the trees.

Another key distinction lies in the learning objective. Random Forest aims to optimize predictive performance by minimizing classification or regression error, whereas Isolation Forest seeks to detect deviations from normal data patterns without requiring labeled examples. As a result, Random Forest is widely used for predictive modeling tasks, while Isolation Forest is particularly suited for anomaly detection scenarios where labeled anomalies are scarce or unavailable. Despite these differences, both algorithms benefit from the ensemble learning framework, which improves robustness and reduces overfitting compared to single decision tree models.

2.9 Hyperparameters

Hyperparameters are configuration parameters that control how a machine learning algorithm learns from data. Unlike model parameters, which are learned automatically during training, hyperparameters must be defined before the training process begins. They influence important aspects of model behavior, including model complexity, training efficiency, and generalization performance.

In ensemble-based algorithms such as Random Forest and Isolation Forest, hyperparameters determine how the individual trees are constructed and how the ensemble operates as a whole. For example, the number of trees in the forest controls the size of the ensemble and can affect both prediction stability and computational cost. Similarly, parameters that limit tree depth or control the number of samples used for training influence the balance between underfitting and overfitting.

Hyperparameters can be initialized using default values provided by machine learning libraries or chosen based on prior empirical knowledge. However, to improve model performance, they are often tuned through systematic optimization techniques. Common approaches include grid search, where a predefined set of parameter combinations is evaluated; random search, which samples parameter values randomly from specified ranges; and more advanced methods such as Bayesian optimization, which iteratively searches for optimal values based on previous results.

For Random Forest models, key hyperparameters typically include the number of trees in the forest, the maximum depth of each tree, and the number of features considered at each split. In Isolation Forest, important hyperparameters include the number of trees, the subsample size used to construct each tree, and the contamination parameter, which estimates the proportion of anomalies in the dataset. Proper tuning of these hyperparameters can significantly improve model accuracy, stability, and anomaly detection capability.

3 Half-Space Trees and Online Anomaly Detection

Data streams have become a vital source of big data. With the advent of the big data era, data streams are generated in various industries as the predominant form of business data. Consequently, data streams have gradually become a research focus in the fields of data mining and information security. Anomaly detection, as an important branch of data mining, aims to identify data patterns that deviate from expected behaviors within data streams. These anomalous patterns often contain crucial information, which is commonly leveraged to assist decision-making processes. Therefore, data stream anomaly detection has emerged as a critical task in data analysis and security domains Bahri et al. (2021).

3.1 Characteristics of Stream Data

Data streams can be seen as data sequences composed of different data items at different times. If t represents any moment in time and x represents the data item at time t , then the data stream can be represented as $x_1, x_2, \dots, x_n$. The length of a data stream is the total number of data items it contains.

In contemporary data science, streaming data is defined as an infinite, sequentially arriving set of observations produced at high velocity. Unlike static datasets, streams are inherently non-stationary, meaning the underlying joint probability distribution $P(X, Y)$ changes over time Fahy, Yang, and Gongora (2022). This non-stationarity manifests primarily as concept drift, where the statistical properties of the target variable such as fraudulent transaction patterns evolve, rendering models trained on historical data obsolete Gama et al. (2014).

Furthermore, streaming environments impose strict memory and computational constraints. Algorithms must adhere to the "single-pass" requirement: each data point is processed once and typically discarded to conserve space. This necessitates a transition from global optimization (common in batch learning) to incremental updates and sliding window mechanisms that prioritize recent history while maintaining a low computational footprint Mitliagkas, Caramanis, and Jain (2013).

3.2 Comparative Analysis of Batch and Stream Data

The following table delineates the fundamental differences between traditional batch (offline) data and stream (online) data, highlighting the shifts in architectural requirements for anomaly detection.

Feature	Batch (Offline) Data	Stream (Online) Data
Data Boundary	Finite and bounded	Infinite and unbounded
Processing Time	High latency (minutes to days)	Low latency (milliseconds)
Data Access	Multiple passes (random access)	Single pass (sequential access)
Memory Usage	High (scales with dataset size)	Low (fixed/constant memory)
Distribution	Assumed stationary (static)	Non-stationary (evolving)
Evaluation	Cross-validation on static sets	Prequential (Test-then-Train)
Complexity	Lower (simple storage/retrieval)	Higher (real-time state management)
Drift Handling	Requires manual retraining	Automated/Adaptive updates
Data Scope	Massive historical datasets	Incoming transaction stream
Accuracy	Generally higher (Deep Learning)	Moderate to High (Speed priority)
Adaptability	Low (Requires retraining)	High (Incremental updates)

3.3 Stream Data Modeling: Advantages and Challenges

The primary advantage of stream modeling is its temporal relevance. By processing transactions in real-time, systems can intervene before a fraudulent event is finalized, minimizing financial loss. Additionally, online models are inherently more efficient for long-term deployments as they do not require the massive storage infrastructure needed to house years of historical "pile" data Arzu et al. (2025).

However, significant challenges persist. Class imbalance is exacerbated in streams; because fraud is rare, a model may "drift" toward a majority-class bias if not carefully calibrated. Moreover, noise in high-velocity streams can be mistaken for anomalies (false positives), causing customer friction. The "evaluation paradox" also arises: since the true label of a transaction may not be known for days, real-time performance monitoring relies on proxy metrics or unsupervised density estimates.

3.4 Common Fraud Detection Patterns and Techniques

In a streaming fraud detection system, success depends on identifying "signatures" of illicit behavior within a narrow window of time. Modern research distinguishes between traditional behavioral patterns (often rule-based) and computational techniques (machine learning) that detect these patterns at scale.

- Velocity Checks: These detect unusually high transaction frequency. If a customer typically makes 2-3 transactions per week but suddenly makes 20 transactions in an hour, the system flags this as suspicious. Implementation uses tumbling or sliding windows to count events over time periods.
- Geolocation Impossibility: Transactions in geographically distant locations within impossible timeframes indicate fraud. The system calculates distance and time between consecutive transactions to identify violations of physical constraints.
- Anomaly Detection: Machine learning models learn normal customer behavior patterns and flag deviations that could indicate fraud. Modern fraud detection systems use algorithms like Isolation Forest (efficient for high-dimensional data), LSTM autoencoders (for detecting temporal sequence anomalies), or one-class SVM (for learning normal behavior boundaries). For example, a customer who always makes small purchases under 50 suddenly attempting a 5,000 transaction would score high on anomaly detection. In 2025, transformer-based models and vector embeddings have become increasingly popular, as they can capture complex patterns across multiple features simultaneously and detect subtle fraud signatures that traditional algorithms miss.
- "Micro-Testing" (Carding): Testing transactions with small amounts followed by large purchases is a common fraud pattern called "card testing."

Small, inconspicuous transactions (e.g., 0.99) made to verify if a card is active before committing a large-scale "cash-out" purchase.

- Network Analysis: Graph processing identifies fraud rings where multiple accounts share devices, IP addresses, or shipping addresses. These network patterns are difficult to detect in isolation but become obvious when analyzing relationship graphs. Graph databases like Neo4j and TigerGraph excel at this analysis, storing entity relationships (accounts, devices, addresses, payment methods) and running graph algorithms like community detection, PageRank, and shortest-path analysis to identify suspicious clusters. Streaming graph processing tools can update these graphs in real-time as new transactions flow, enabling dynamic fraud ring detection.
- Behavioral Divergence: Sudden shifts in merchant categories, such as a user who typically buys groceries suddenly making high-end jewelry or electronics purchases.
- Statistical Outlier Detection: This uses mathematical distributions (like Z-scores or Mahalanobis Distance) to find data points that fall outside the "normal" bell curve of spending. Technique: Comparing a new transaction's value against the user's historical mean and standard deviation.
- Graph-Based Analysis: Advanced systems use Graph Neural Networks (GNNs) to detect collusion. For example, if ten different accounts all send money to the same "mule" account within an hour, the graph structure reveals a co-ordinated attack that single-row analysis would miss.

3.5 Why Offline Isolation Forest Struggles

The Isolation Forest (iForest) is great at finding "lonely" data points (anomalies). However, the standard version (Scikit-learn) has flaws in streaming:

- It is Static: It builds a "forest" based on a snapshot of the past. If a new type of fraud appears, the forest is "blind" to it.
- High Latency: To update the model, you have to stop the system, add new data to the old data, and rebuild the entire forest from scratch.
- Memory Heavy: It requires keeping large amounts of historical data to perform well.

3.6 Online Isolation Forest

Since fraud is rare and labels are often missing in real-time, unsupervised techniques like Isolation Forest and Half-Space Trees are preferred. Instead of modeling "normal" behavior, these techniques try to isolate outliers. Because fraudulent transactions are "few and different," they require fewer random splits to be separated from the rest of the data

The standard Isolation Forest builds a set of trees (*iTrees*) by randomly selecting a feature and a split value. It assumes the data you used to build those trees represents the world forever. Online Isolation Forest (OIF) breaks this assumption. It treats the model as a living structure that must update as each transaction arrives. The primary goal is to maintain the "Isolation" principle—that anomalies are easier to partition—even when the definitions of "normal" and "anomalous" are shifting.

How it Works (The Streaming Mechanism): While there are different versions of OIF, the implementation in streaming libraries like River typically follows these mechanics:

- Window-Based Replacement: The forest is not rebuilt. Instead, individual trees within the forest are "retired" based on their age or performance and replaced by new trees trained on the most recent "window" of data.
- Incremental Mass Updates: Rather than changing the physical structure of a tree for every point (which is computationally expensive), the model updates the "count" or "mass" of the nodes. If a new transaction falls into a leaf that previously had 1,000 points, that count becomes 1,001.
- Path Length Calculation: The anomaly score is still derived from the path length. However, in an online setting, the "expected path length" is dynamically adjusted based on the current size of the window, not the original training set.

Concept Drift: OIF handles drift through Tree Partial Resetting. If the forest detects that the average path lengths are globally increasing (meaning the trees no longer "isolate" points well), it triggers a refresh of the oldest trees. This ensures the model "forgets" obsolete fraud patterns from months ago. Types of Drift in Fraud Detection are given below:

- Sudden Drift: Rapid changes, such as a coordinated bot attack introducing a new transaction pattern overnight.
- Gradual Drift: Long-term shifts in consumer behavior, such as the slow transition toward digital wallet adoption over several years.
- Incremental Drift: Subtler changes where a fraudster "warms up" a stolen card with minor increases in transaction amounts to avoid triggering detection limits.

The Sliding Window Strategy OIF does not maintain a permanent memory of the entire stream. Instead, it operates on a moving temporal window. This ensures that the model's definition of "isolation" is derived from recent data distributions. As the window slides, older observations lose their influence on the tree's mass, effectively "forgetting" obsolete patterns that no longer represent current spending habits.

Reference vs. Latest Window Estimation The implementation of streaming isolation in the RiverML library utilizes a dual-window approach to detect and adapt to drift:

- Window (W_r): Stores the "mass" (node counts) of a stable historical period, representing the baseline for normal activity.
- Latest Window (W_l): Accumulates counts from the current, incoming stream of transactions.
- The Adaptive Switch: Once W_l reaches a predefined size (the window limit), the model compares its density against W_r . If a significant distribution shift is detected, the model performs a structural update where W_l becomes the new W_r , and the latest counts are reset. This allows the model to re-calibrate what constitutes a "low-mass" area (anomaly) in real-time.

ADWIN: Automated Drift Detection While OIF has internal adaptation, higher-level systems often integrate an external monitor called ADWIN (Adaptive Windowing). ADWIN tracks the global anomaly score and utilizes a Hoeffding Bound to detect statistically significant changes in the average score. When a "Drift Event" is signaled, the model can respond by accelerating the replacement of outdated trees or adjusting its learning rate to accommodate the new data distribution.

3.7 Key Components of a Streaming Fraud Detection System

The following are the key components of a robust streaming fraud detection architecture:

- Data Ingestion and Message Brokerage: The ingestion layer serves as the entry point for raw transaction events. Utilizing technologies like Apache Kafka or AWS Kinesis, this component acts as a high-throughput buffer. Its primary role is to decouple data sources from the processing engine, ensuring that the system is fault-tolerant and capable of handling sudden spikes in transaction volume without losing data.
- Stream Processing and Incremental Feature Engineering: Because streaming data is processed in a single pass, features must be calculated "in-flight." This layer transforms raw transaction data into meaningful metrics, such as "number of transactions in the last hour" or "average spend over the last 10 minutes." Using frameworks like River or Apache Flink, the system maintains a running state to perform online scaling and normalization, which is essential for models like Half-Space Trees.
- Online Inference Engine: This is the core analytical component where the machine learning model (e.g., Online Isolation Forest or Half-Space Trees) resides. Unlike offline models that provide scores based on historical snapshots, the online engine scores each transaction the moment it arrives. It

identifies anomalies by detecting "low-mass" events—transactions that deviate significantly from the current behavioral distribution of the stream.

- Real-Time Feature Store and Contextual Cache: For a model to determine if a transaction is fraudulent, it often needs historical context about the specific user. A real-time feature store (such as Redis) provides low-latency access to pre-calculated user profiles. This allows the inference engine to compare the current transaction against the user's "normal" historical behavior in milliseconds.
- Adaptive Feedback and Learning Loop: A critical component of streaming systems is the ability to adapt to concept drift. This layer monitors model performance and updates internal parameters based on the most recent data windows. By utilizing a "Test-then-Train" (prequential) approach, the system continuously refines its logic, ensuring it learns from new fraud patterns as they emerge while "forgetting" obsolete data.
- Real-Time Alerting and Orchestration: Once a transaction is scored, the system must execute a decision. If the anomaly score exceeds a certain threshold, the orchestration layer triggers automated actions, such as declining the transaction, flagging it for manual review, or initiating "step-up" authentication (e.g., an SMS OTP). This ensures that detection translates directly into loss prevention.

3.8 Key Components of the River Architecture

River is an open-source Python library specifically designed for online machine learning on dynamic data streams. It was formed through the merger of two prominent libraries: Creme (focused on single-instance learning) and scikit-multiflow (focused on stream mining algorithms). Three structural pillars of River that make it superior for streaming fraud detection:

- The `learn_one` / `predict_one` Interface: Standard ML libraries use `.fit(X, y)` for entire datasets. River uses `learn_one(x, y)` and `score_one(x)`. This allows the model to perform Prequential Evaluation (test-then-train): the system predicts the fraudulence of a transaction before it uses that same transaction to update its internal logic.
- Streaming Pipelines: River allows us to "chain" operations together using a syntax similar to Unix pipes (e.g., `Scaler | Model`). Crucially, the Standard-Scaler in River is also incremental; it maintains a running mean and variance rather than calculating them from a fixed dataset.
- Dictionary-Based Processing: While Scikit-learn relies on NumPy arrays (which are fast for batches), River uses Python dictionaries. This is a strategic design choice: dictionaries allow for feature evolution, where new types of data fields can appear in the stream without breaking the model—a common occurrence in evolving financial datasets.

3.9 Half-Space Trees

Half-Space Trees, introduced by Tan, Ting, and Liu (2011), are a set of online, unsupervised anomaly detection algorithms designed to approximate the "Isolation" principle in a streaming context. While the standard Isolation Forest uses random splits and path lengths, HSTs use a fixed-tree structure and "mass" (point frequency) to identify outliers.

Fixed Tree Architecture: Unlike traditional trees that grow based on data, an HST is pre-defined.

- Initialization: During a "warm-up" phase, the algorithm determines the maximum and minimum values for each feature in the dataset to establish a bounding box.
- Random Partitioning: The tree is constructed to a fixed depth (h). At each node, a feature is randomly selected, and the space is split exactly in half. This creates a set of "half-spaces" (cubes or hyper-rectangles) in the feature space.
- Computational Constant: Because the structure is fixed, the time complexity to process a transaction is $O(h)$, ensuring the system remains high-velocity.

The Mass-Counting Logic The fundamental theory of HST is that anomalies occupy regions of low "mass."

- Mass Assignment: As a transaction streams in, it "traverses" the tree based on its feature values until it reaches a leaf node. Every node along that path increments its counter (Mass).
- Density Estimation: An anomaly is defined as a point that falls into a branch of the tree where the total mass is significantly lower than the global average. In a fraud context, a "normal" transaction pattern will have a high mass (many similar transactions), while a "fraudulent" pattern will have a very low mass.

Theoretical Changes for Streaming To make the Isolation Forest logic work in an unbounded stream, HST introduces two critical changes:

- Mass Decay (The Reference Window): To handle Concept Drift, HST maintains two counters: a Reference counter and a Latest counter. Once a certain number of points are seen, the Latest counts replace the Reference counts. This allows the model to "forget" old spending patterns and adapt to new ones.
- Normalized Scoring: The anomaly score is normalized between 0 and 1 based on the maximum possible mass in the tree, allowing for consistent thresholding in Streamlit visualizations.

References

- Aggarwal, Charu C. (2017). *Outlier Analysis*. 2nd. Cham, Switzerland: Springer International Publishing. ISBN: 978-3-319-47577-6. DOI: 10.1007/978-3-319-47578-3.
- Arzu, Farah et al. (2025). “Real-Time Financial Fraud Detection: An Intelligent Data-Driven Framework Integrating Machine Learning, Stream Processing, and Big Data Analytics for High-Velocity Transaction Monitoring”. In: *The Asian Bulletin of Big Data Management* 5.4, pp. 124–154.
- Bahri, Maroua et al. (2021). “Data stream analysis: Foundations, major tasks and tools”. In: *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 11.3, e1405.
- Barnett, Vic, Toby Lewis, et al. (1994). *Outliers in statistical data*. Vol. 3. 1. Wiley New York.
- DeepFA (Oct. 2, 2025). *Isolation Forest algorithm: Anomaly detection with machine learning*. URL: <https://deepfa.ir/en/blog/isolation-forest-algorithm-anomaly-detection-machine-learning>.
- Fahy, Conor, Shengxiang Yang, and Mario Gongora (2022). “Scarcity of labels in non-stationary data streams: A survey”. In: *ACM Computing Surveys (CSUR)* 55.2, pp. 1–39.
- Gama, João et al. (2014). “A survey on concept drift adaptation”. In: *ACM computing surveys (CSUR)* 46.4, pp. 1–37.
- Grubbs, Frank E (1969). “Procedures for detecting outlying observations in samples”. In: *Technometrics* 11.1, pp. 1–21.
- Hawkins, D (1980). *Identification of outliers*.
- Kavlakoglu, E. (2026). *What is Random Forest?* IBM Research. URL: <https://www.ibm.com/topics/random-forest> (visited on 03/06/2026).
- Mitliagkas, Ioannis, Constantine Caramanis, and Prateek Jain (2013). “Memory limited, streaming PCA”. In: *Advances in neural information processing systems* 26.
- Tan, Swee Chuan, Kai Ming Ting, and Tony Fei Liu (2011). “Fast anomaly detection for streaming data”. In: *IJCAI proceedings-international joint conference on artificial intelligence*. Vol. 22. 1, p. 1511.